Formal Languages and Automata

Zhengfeng Ji, *Department of Computer Science and Technology, Tsinghua University*

1 About This Course

1.1 Welcome

This is an incomplete set of notes prepared by Zhengfeng Ji for the course. Please do not distribute.

1.2 Formal languages and automata

1.3 Lecturer

Zhengfeng Ji

Office: 1-707, Ziqiang Technology Building

Email: jizhengfeng@tsinghua.edu.cn

Research areas: quantum computing and theory of computing.

- Teaching assistants

  Huiping Lin (lhp22@mails.tsinghua.edu.cn)

  Shenghan Gao (sh-gao24@mails.tsinghua.edu.cn)

- This is the fourth time I have taught this course at Tsinghua.
- Your feedback is welcome

1.4 Computation and mathematics

- Also known as Introduction to the ToC in some universities
- Key observation

  Computation, Mathematics, Logic

  Computational problems, devices, and processes can be viewed as mathematical objects.

  One interesting connection between computation and mathematics, which is particularly important from the viewpoint of this course, is that mathematical proofs and computations performed by the models we will discuss throughout this course have a low-level similarity: they both involve symbolic manipulations according to fixed sets of rules. (Watrous lecture notes)

  We will study different types of recursions, ***Kleene*** star, context-free grammars, and Turing machines.

1.5 What you will learn in this subject

1. Finite automata and regular expressions
2. Pushdown automata and context-free grammars (CFG)
3. Turing machines and computability, and basic complexity theory

- More importantly, we hope that you will learn how to write definitions and proofs rigorously and improve your mathematical maturity.
- I offer a follow-up course next semester called ***Introduction to Theoretical Computer Science***, which covers lambda calculus, advanced computability, complexity theory, probabilistic computation, cryptography, and propositions as types.

  See lecture notes at http://itcs.finite-dimensional.space

1.6 Teaching and learning

- We use chalk and a blackboard.

  The chalk-and-blackboard method is better for teaching and learning theoretical subjects.

  In How to Speak, Patrick Winston (MIT) argues that a blackboard is the right tool for speaking when the purpose is to inform.

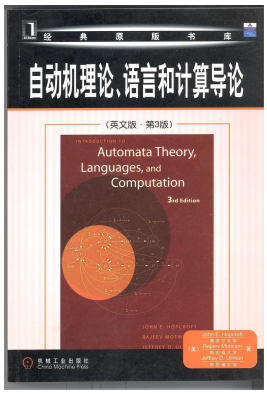
- Please be actively involved!
- Reviewing after each class is highly recommended.
- We will hand out notes after each class for you to review the material covered in class.
- The subject could be challenging for many of you.

  It has a different style and a different set of requirements.

- Pause and think about the problem or theorem by yourself before looking at the proof.

> *Barak: Even a five-minute attempt to solve the problem by yourself will significantly improve your understanding of the topic.*

1.7 Textbook



Introduction to Automata Theory, Languages, and Computation (3rd ed.), by John Hopcroft, Rajeev Motwani, and Jeffrey Ullman.

Hopcroft: In 1986, he received the Turing Award (jointly with Robert Tarjan) for fundamental achievements in the design and analysis of algorithms and data structures. He helped to build research centers at Shanghai Jiao Tong University and Peking University.

Motwani: He received the Gödel Prize for the PCP theorem and was the author of another excellent textbook, ***Randomized Algorithms***. He was the youngest of the three, but unfortunately, he passed away in 2009.

Ullman: He received the Turing Award (2020) for his work on compilers and was an author of the Dragon Book, ***Compilers: Principles, Techniques, & Tools***. He was the Ph.D. advisor of Sergey Brin, one of the co-founders of Google.

- Other materials:
  1. Introduction to the Theory of Computation, Michael Sipser
  2. Watrous lecture notes (https://cs.uwaterloo.ca/~watrous/ToC-notes/)
- Lecture notes: http://fla.finite-dimensional.space

1.8 Homework

There will be about 12 assignments.

You are strongly encouraged to complete them on time.

1.9 Grading

Mid-term: 25%

Attendance & homework: 15%

Final exam: 60%

1.10 Q & A

- Online: WeChat group, learn.tsinghua.edu.cn, and email
- Office hours: 4–5 p.m. on Tuesday

2 Why Study This Subject

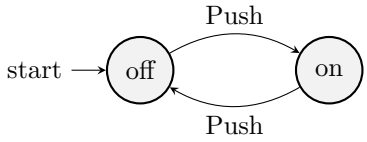
Classic material (mostly developed between the 1930s and the 1960s)

2.1 Finite automata

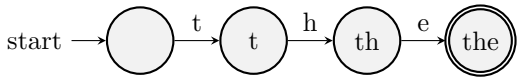
- Simple examples of finite automata

  This simple model enables a deep understanding of its properties and the development of efficient algorithms for manipulating it.

  Example 1. Figure 1.1: Push a button to switch between on and off, starting from the off state.



  Example 2. Search for the word **the**.



- Finite automata are a valuable model for many important kinds of hardware and software.
  - Specification, verification, and model checking of systems of finite states, circuits, and communication protocols
  - Lexical analyzers for compilers
  - Text search
- When and how were finite automata discovered?

  [McCulloch and Pitts, A logical calculus of ideas immanent in nervous activity, 1943]

  [Kleene, Representation of events in nerve nets and finite automata, 1956]

  [Rabin and Scott, Finite automata and their decision problems, 1959]
- Regular expressions

  $[A-Z][a-z]*$ matches Computer

2.2 Context-free grammars

- Grammars are useful models when designing software that processes data with a recursive structure.

  Programming languages

  Compilers never enter an infinite loop.
- Pushdown automata

  A machine with a single stack.

2.3 Turing machine

- What can a computer do at all?

  Hilbert's 1928 Entscheidungsproblem (cf. Hilbert's tenth problem about Diophantine equations)

  The problem asks for an algorithm that takes a statement as input and answers **Yes** or **No** according to whether the statement is valid.

  What cannot be done on a computer?

  - We must know. We will know.

    The epitaph on his tombstone in Göttingen consists of the famous lines he spoke after his retirement address to the Society of German Scientists and Physicians on 8 September 1930.

- **What is computation?**

  Church: Lambda calculus ($\lambda x.x^2 + 2$)

  Gödel: Recursive functions

  Turing: Turing machines (age 24)

  You: C, Python, Java, …

- The Turing machine is an intuitive model of computation, and Gödel was convinced by it.

  It's like waiting for the bus. We waited for 2000 years for a formal definition of computation; three buses come along at the same time.

- They are all equivalent. **Is mathematics invented or discovered?**
- **Church-Turing thesis**
- Logic and computation

  Gödel's incompleteness: The first incompleteness theorem states that no consistent system of axioms whose theorems can be listed by an effective procedure (i.e., an algorithm) is capable of proving all truths about the arithmetic of natural numbers.

- What can a computer do efficiently?

  Space/time-bounded TMs

- **Extended Church-Turing thesis**

3 Basics of FL&A

3.1 Sets and countability

Naive set theory treats the concept of a set as self-evident. This will not be problematic for this course, but it does lead to problems and paradoxes—such as Russell's paradox—when it is pushed to its limits.

Bertrand Russell and Russell's paradox. Let S be the set of all sets that are not elements of themselves: $S = \{T : T \text{ is not in } T\}$. Is it the case that S is an element of itself? (***Self-reference***)

The size of a finite set S is denoted by $|S|$.

Sets can also be infinite.

Definition. A set A is countable if either (i) A is empty, or (ii) there exists an onto (or surjective) function of the form $f : \mathbb{N} \to A$. If a set is not countable, we say that it is uncountable.

Power sets:

- 2^A or $P(A)$
- More generally: What is A^B?

Theorem. The set $\mathbb{Q}$ of rational numbers is countable.

Theorem (Cantor). The power set of the natural numbers, $P(\mathbb{N})$, is ***uncountable***.

There is a technique at work in this proof known as **diagonalization**. It is a fundamentally important technique in the theory of computation, and we will see instances of it later.

3.2 Central concepts of formal languages

1. Alphabets

   An alphabet is a finite nonempty set of symbols.

   $\Sigma = \{0, 1\}$

   $\Sigma = \{a, b, c, \ldots, z\}$

2. Strings

   Strings are sequences of symbols chosen from an alphabet.

   0101 is a sequence over the alphabet $\{0, 1\}$.

   The empty string: ϵ

   The length of a string: $|010| = 3$

   Powers of an alphabet:

   Σ^3

   The set of all strings over an alphabet: Σ^*

   $\Sigma^* = \cup_{n \geq 0} \Sigma^n$

   Σ^+

   Concatenation xy

3. Languages

   A language is a subset L of Σ^*.

   Set-builder notation provides a way to define languages.

   For example, consider the set of binary numbers whose values are prime.

4. Problems

   In automata theory, a problem is the question of deciding whether a given string is a member of some particular language.

5. Union/concatenation/closure (star, Kleene closure)

   $L \cup R$

   LR

   $L^* = \bigcup_{i=0}^{\infty} L^i$

3.3 Proof techniques

- Proofs about sets

  Ex. $R \cup (S \cap T) = (R \cup S) \cap (R \cup T)$

- Proofs by contradiction
- Proofs by induction

3.3.1 Inductive proofs

Prove a statement $S(n)$ for every integer $n \geq n_0$.

Basis: $S(n_0)$ is true.

Induction: If $S(n)$ is true, then $S(n+1)$ is true for $n \geq n_0$.

3.3.2 Mutual induction

Prove statements together!

Example. In the on-off automaton,

$S_1(n)$: The automaton is in the off state after n button pushes if and only if n is even.

$S_2(n)$: The automaton is in the on state after n button pushes if and only if n is odd.

3.3.3 Structural induction

The most important proof technique for automata theory and programming languages.

Ex. Trees

Basis: A single node is a tree, and that node is the root of the tree.

Induction: If $T_1, T_2, \ldots, T_k$ are trees, then we can form a new tree as follows:

1. Begin with a new node N, which is the root of the new tree.
2. Add copies of all the trees $T_1, \ldots, T_k$.
3. Add edges from node N to the roots of each of the trees $T_1, \ldots, T_k$.

Theorem. Every tree has one more node than it has edges.

Proof. Use structural induction. Basis. The tree has a single node. Induction. Assume the claim is true for $T_1, \ldots, T_k$, …

Ex. Expressions

Basis: Any number or letter is an expression.

Induction: If E and F are expressions, then so are $E + F$, $E * F$, and (E).

4 Reading

- Watch Veritasium: Math's Fundamental Flaw